Associate Degree in Software Engineering
with focus on Web Programming
June 2026



Accessibility Across the Shadow Boundary

Bridging WAI-ARIA APG and Shadow DOM Implementation Gaps

Simon Darsell
sidr24@student.bth.se

Abstract

This study investigates how accessible Web Components can be implemented using current web standards despite the accessibility limitations introduced by the Shadow DOM. While the encapsulation provided by Shadow DOM improves modularity and reusability, it creates a DOM boundary that prevents WAI-ARIA relationships and interaction patterns from working as intended. This presents a challenge for developers seeking compliance with the Web Content Accessibility Guidelines (WCAG) 2.1 AA and to follow the European Accessibility Act (EAA).

Using a Design Science research approach, three WAI-ARIA Authoring Practices Guide (APG) patterns - Modal Dialog, Combobox, and Checkbox - were implemented as Shadow DOM-based Web Components and evaluated through automated accessibility testing and manual testing with keyboard navigation and screen reader. The study identifies accessibility challenges related to cross-boundary ID references, focus management, form participation, and browser inconsistencies.

The findings show that APG patterns are conceptually transferable to Shadow DOM environments, but direct implementation often fails due to assumptions of a single DOM tree. However, this study identifies implementation strategies such as attribute forwarding, Shadow-host delegation, ElementInternals, and reflected element-reference APIs. These strategies can help developers to address the accessibility barriers Shadow DOM introduces and support developers in meeting WCAG 2.1 AA requirements.

Table of Contents

1. Introduction 5
    1.1 Background 5
    1.2 Problem Statement 5
    1.3 Motive and Value 5
    1.4 Focus Areas 6
        This study focuses on: 6
    1.5 Research Questions 6
        RQ1 6
        RQ2 6
        RQ3 6
    1.6 Limitations 6
2. Related Work / Literature study 7
    2.1 Literature Gap 7
3. Method 7
    3.1 Research approach 7
        3.1.1 Observational 8
        3.1.2 Analytical 8
        3.1.3 Experimental 8
        3.1.4 Testing 9
        3.1.5 Descriptive 9
    3.2 Component Design Considerations 9
4. Results and Analysis 10
    4.1 Modal Pattern 10
        4.1.1 Requirements for Modal Pattern 10
        4.1.2 Implementation of Modal Pattern 10
        4.1.3 Evaluation of Modal Pattern 11
    4.2 Implementation of Combobox Pattern 14
        4.2.1 Requirements for Combobox Pattern 14
        4.2.2 Implementation of Combobox Pattern 14
        4.2.3 Evaluation of Combobox Pattern 15
    4.3 Checkbox Pattern 17
        4.3.1 Requirements for Checkbox Pattern 17
        4.3.2 Implementation of Checkbox Pattern 17
        4.3.3 Evaluation of Checkbox Pattern 18
    4.4 Pattern Analysis 20
        4.4.1 RQ1 - How does the selected WAI-ARIA APG patterns work when implemented in Shadow DOM-based Web Components? 20
        4.4.2 RQ2 - Which accessibility problems occur when working with shadow DOM encapsulation, what are the technical causes? 21
        4.4.3 RQ3 - Which implementation strategies and workarounds can be used to support WCAG 2.1 AA compliance for Web Components when APG patterns can not be directly applied? 21

5. Discussion 22
5.1 The Consuming Developer 22
5.2 EAA Compliance 22
5.3 Limitations of the Study 23
6. Conclusion & Future Work 24
References 25
Appendices 28
Appendix A: 28
Testing environment 28

1. Introduction

1.1 Background

To comply with the European Accessibility Act [1], active since 28 June 2025, services like e-commerce, booking and online banking are required to follow the EN 301 549 standard, largely derived from WCAG 2.1 [2] defined by World Wide Web Consortium (W3C).

There are educational guides from W3C like WAI - ARIA Authoring Practices Guide [3] that developers can use to assist them, but they, as many other parts of the web, are created for the standard HTML elements, and on the assumption of one, light, DOM.

Web Components and the Shadow DOM are more and more common, statistics show substantial use of Shadow DOM although these numbers include browser-internal usage [4]. Despite this evolution, developers lack not only guides, but sometimes actual methods to work in an accessible way with the shadow DOM.

1.2 Problem Statement

The encapsulated nature of Web Components is the reason they have gained in popularity, but it creates a DOM barrier where for example IDs from one DOM can not be referenced in the other, many relations and ARIA-attributes are based on those IDs.
This results in developers making workarounds to force custom elements to adhere to accessibility standards.

Meanwhile W3C is constantly working on new features[5], and the ongoing development of Cross Root ARIA[6] which aims to solve some of the limitations could have a great impact on this field, if and when it happens. Meanwhile there is a gap between the accessibility requirements as the ones introduced by EAA and the developer guides and technical options to implement them, which this study aims to address.

1.3 Motive and Value

I intend to investigate how established accessibility patterns can be implemented as Shadow DOM-based Web Components using the current web standards. I will particularly focus on the accessibility issues introduced by Shadow DOM and examine practical implementation strategies to work around them.

Accessibility is not just a legal requirement in some contexts, it is also a cornerstone for an inclusive digital world. Seeing the lack of research in this topic and developers asking for help on Stack Oveflow[7] (just as an indicator, not a source of truth) shows this is an underresearched area. This study therefore aims to contribute both observations of Shadow DOM accessibility behaviour and practical implementation strategies that can be useful to other developers working with accessible Web Components.

1.4 Focus Areas

This study focuses on:

- **Accessibility in Web Components** - Analysis of how accessibility requirements and interaction patterns are affected when implemented as Shadow DOM Web Components.
- **Shadow DOM and encapsulation** - How does the encapsulation affect accessibility, relations, focus order and ARIA-attributes.
- **Developer practices** - Identify and analyze current solutions for achieving accessibility in Shadow DOM.
- **Analysis of WAI-ARIA APG in context of the Shadow DOM** - Evaluate and adapt a few selected WAI-ARIA APG patterns to work with the Shadow DOM.

1.5 Research Questions

RQ1

How does the selected WAI-ARIA APG patterns work when implemented in Shadow DOM-based Web Components?

RQ2

Which accessibility problems occur when working with shadow DOM encapsulation, and what are the technical causes?

RQ3

Which implementation strategies and workarounds can be used to support WCAG 2.1 AA compliance for Web Components when APG patterns can not be directly applied?

1.6 Limitations

This study does not intend to include:

- A full component library.
- Browser compatibility beyond modern versions of Chrome, Safari and Firefox.
- Mobile-specific accessibility.
- Testing performed with assistive technologies equipment like refreshable braille display.
- User testing by experienced screen reader-users.
- Exhaustive WCAG 2.1 AA testing for all applicable criteria.
- The study evaluates three selected APG patterns and findings can not be generalized to all APG patterns

2. Related Work / Literature study

Despite the increased use of Web Components in recent years very little has been written academically on how accessibility patterns can be implemented within the shadow DOM. Most writings on this topic are in blog posts, GitHub-issues and forums rather than in academic work.

A.A. Gregory [8] examines accessible Web Components with a focus on the correct application of ARIA attributes and semantic HTML within custom elements. While the work provides a useful foundation for understanding accessibility in the Web Components context broadly, it does not address the specific challenges introduced by Shadow DOM

In Meeting Accessibility Challenges with Web Components, White, Hakkinen and Grant [9] describe scenarios where Web Components can be used to improve accessibility, for example by encapsulating complex interaction patterns into reusable elements. The shadow DOM is acknowledged but not examined as a source of accessibility problems, and the study does not address the DOM boundary constraints that this study investigates.

Parab [10] briefly touches on Shadow DOM encapsulation issues in the context of a broader overview of Web Components, referencing practitioner work by Lawson [11]. Lawson provides a detailed analysis of how Shadow DOM encapsulation creates friction for developers, including limitations around ARIA ID references across DOM boundaries. While not academic, Lawson's analysis represents the kind of practitioner knowledge that currently fills the gap left by formal research, and the problems it identifies are directly relevant to this study.

Across the academic works, the Shadow DOM accessibility problem is either absent or only briefly acknowledged. This study address that gap directly by systematically implementing and evaluating WAI-ARIA APG patterns in a Shadow DOM context, with the aim of producing empirical findings that complement the existing practitioner literature.

2.1 Literature Gap

I did not find any academic writing directly on the problems with Shadow DOM and the accessibility problems that are introduced by the Shadow DOM encapsulation.

However, when looking at web conferences and talks[12], discussions in W3C Web Component working groups[13], and blog posts from experienced developers it shows a strong indication that there is a research gap in the academic literature.

3. Method

This chapter describes the research methodology, implementation strategy, and testing procedures used in the study.

3.1 Research approach

This study is performed with a Design Science approach as described by Hevner et al.[14] where an artifact is designed, tested and the results are evaluated. The artifact consists of a set of Web Components built with Shadow DOM. The components are intended to follow the WAI-ARIA APG patterns and investigate implementation strategies that can support compliance with WCAG 2.1 AA guidelines. Design Science was selected because it allows iterative comparative evaluation of different design solutions and refinements. The research

process is structured into five phases: observational, analytical, experimental, testing and descriptive.

3.1.1 Observational

The study begins with a literature review to establish the theoretical and technical foundations of the problem. Primary sources include the WAI-ARIA Authoring Practices Guide (APG), the WCAG 2.1 specification, the HTML and Shadow DOM specification. These documents define both the relevant accessibility requirements that the implementations should meet and the technical constraints and functionality of the Shadow DOM environment.

The literature review also covers related academic work and practitioner sources as described in chapter 2, in order to map the existing knowledge and investigate the research gap.

3.1.2 Analytical

To better understand the field, current implementations of Shadow DOM-based Web Components in established libraries like Shoelace, Adobe Spectrum, Web Awesome were examined to gain insight in strategies regarding the DOM-boundary and accessibility.

Based on the literature review and the analytical part I have selected three WAI-ARIA APG patterns that present a broad set of DOM boundary related accessibility challenges. The selected patterns require cross boundary ID references in both directions, form association from inside the Shadow DOM, focus management and management of ARIA-states across the DOM boundary. The patterns selected for this study are: Modal Dialog, Combobox and Checkbox.

Modal Dialog is a form of dialog window where the underlying content is inactivated, usually displayed with the background visually blurred or covered with a semi transparent layer. This is commonly used for important information or to prompt the user for decisions before further interactions in a web site or -application. The pattern requires focus to be moved to the dialog in Shadow DOM when opened, stay there during interaction and return to the invoking element in the Light DOM when it closes.

A Combobox is an input that controls a popup list of values for the end-user to select from. Comboboxes can be editable, in this case the user can narrow down the selectable values based on keyboard input. In a Shadow DOM context it is likely that the developer using the component wants to insert the selectable values through a slot, this will create several reference problems over the DOM boundary.

Checkbox is a dual-state toggle, most commonly used within a form. The pattern requires proper grouping, labelling, and often form association, so that the component's value can be submitted as part of a form element in the light DOM while keeping its internal implementation in Shadow DOM.

3.1.3 Experimental

The artifact is implemented using ECMAScript, HTML and CSS, no build tools or transpilers were used, all to keep the experiment as unobfuscated and close to the web platform as possible. This makes errors and side effects as easy to trace as possible and the results easier to analyse. Each component was created iteratively, with accessibility testing performed

continuously during the implementation. The requirements for each component were based on the corresponding WAI-ARIA APG pattern. The official APG examples were used as implementation references.

The project repository was used to preserve the implementation history through the Design Science process. Earlier implementation attempts, and its refinements can be traced together through commit history together with test outputs from that stage to increase traceability.

3.1.4 Testing

Each implemented component was tested using axe-core through the Playwright framework and Google's Lighthouse, and manual testing was performed using Apple VoiceOver in combination with Chrome, Firefox and Safari and keyboard navigation.

Playwright was used to verify interaction behaviour such as opening and closing components, keyboard navigation, focus movement, roles, states, state changes and form participation. Axe-core and lighthouse were used as static accessibility tests to find structural errors, but also problems related to roles, states and other accessibility properties.

Manual testing was used as a complement to assert that the implementation could be operated and perceived by a human user. It was also used for parts that are harder to test automatically like screen reader announcements, focus behaviour and browser specific differences in assistive technology-output.

Tests and their results are available in the repository and were updated along with the iterations of the design science process.

3.1.5 Descriptive

The findings from the experimental and testing phases were analysed and compared across the three implemented patterns. The analysis focused on finding recurring Shadow DOM-related accessibility problems, their technical cause and implementation strategies to address them.

The results were used to evaluate which strategies were successful, which introduced other problems or shifted developer responsibility, and which problems could not be solved using the original APG approach.

3.2 Component Design Considerations

Creating a Web Component usually implies reusability, modularity and flexibility. When designing the artifacts the assumption was that they should be reusable and dynamic, in the sense that different content should be able to be passed into the components by a consuming developer. This means the components were designed to accept slotted content from the Light DOM rather than having their content hardcoded inside the Shadow DOM. This design decision is realistic and representative of how Web Components are used in practice, but it also heightens exposure to the DOM boundary challenges that this study investigates, since slotted content lives in the Light DOM while the component logic lives in the Shadow DOM. Another common use case could however be a more static component, to be reused across an enterprise's webpage, or that could be used on any website, but maintain the exact design and functionality wherever it is placed. I will not examine that use case in depth.

4. Results and Analysis

4.1 Modal Pattern

4.1.1 Requirements for Modal Pattern

Requirement area	Requirements
Role	Dialog container has role=”dialog”
Modal State	Dialog is modal - page content outside of the dialog is not possible to interact with.
Accessible name	Dialog has an accessible name
Initial focus	Focus moves to an appropriate element inside dialog when opened
Keyboard	Tab and Shift + Tab cycles through focusable elements without reaching underlying page
Closing	Escape closes the dialog, a visible close button is available. Focus restores to the invoking element
Study-specific Shadow DOM requirement	
Slotted initial focus	The initial focus target can be slotted from the Light DOM

Table 4.1.1.1 Requirements for the Modal pattern

The implementation used the native Dialog element[15] and the Invoker Commands API[16] they are considered Baseline and have good browser support.

Note: the APG states that tabbing should loop inside the modal and not leave the modal container, however WCAG success criteria 2.1.2[17] states that the user agent (browser controls) could be included in the loop. There is an ongoing debate[18] about what is the correct approach and different browsers implement it in different ways. My implementation relies on the native behaviour of the dialog and the user’s browser, if consensus is reached the browsers will change their focus loop, my code will automatically follow that by using a dialog element with *show-modal* method.

4.1.2 Implementation of Modal Pattern

The component [19] uses a slot for the content of the modal where a consuming developer could set their content of choice. The usage of the semantic dialog element allows the possibility to use its showModal-method. This method opens the dialog in a modal-mode which makes the rest of the page inert and traps focus according to the latest web standards.

With the Invoker Commands API [20] I can add a *commandfor*-property to a button,

this way I can send it a command, in this case the command show-modal which is the declarative way of running the openModal-method, this lets us use the capabilities already built in to the HTML elements and will be more future proof.

However, there is a Shadow DOM related problem here: The commandfor attribute value must match the ID of the target element but since the modal we want to open is in the Shadow DOM we can not reference its ID from the button in the Light DOM. This highlights the problem of interacting over the DOM boundary, and causes my initial implementation to fail by not opening the dialog.

To address this problem I pass the command to the Web Component that is the Shadow Host and from there delegate it to the dialog inside the Shadow DOM, however I can not use the string show-modal since it is a reserved value and instead assign –show-modal as a custom command. Inside the web component I add an event listener that listens for that custom command and programmatically invokes the dialog-elements showModal-method. The same method is used to give the modal a label, the component has a modal-label attribute where the user can add a description that is programmatically added to the dialog element.

The APG pattern requires that focus moves to the dialog when it opens, ideally to the first focusable element. Since the dialog content is provided via a slot, the focusable elements live in the light DOM. The native delegatesFocus option on the attachShadow method is designed to delegate focus into the shadow DOM-tree, but it only reaches elements inside the shadow DOM itself, not slotted light DOM content. This means that focus lands on the close button at the bottom of the modal dialog, which is the first focusable element inside the shadow tree, rather than on the first focusable element provided by the developer, this is the case for Safari and Firefox, however Chrome manages to set focus on a slotted focusable element.

Examining the source code of the Web Awesome component library [21] revealed a workaround; querying the shadow host for an element with the autofocus attribute and setting focus on it manually. Since the querySelector on the host element searches the light DOM, it successfully crosses the shadow boundary in the right direction. This works consistently across browsers but places more responsibility on the consuming developer using the component to mark which element should receive focus, in cases where focus would normally automatically be set, like buttons or inputs the consuming developer would still need to set an autofocus attribute. This is not automatic fulfillment of the APG Pattern but a reasonable workaround, autofocus is a standard HTML attribute with exactly that semantic meaning.

4.1.3 Evaluation of Modal Pattern

In the final state of the component the host is a functional wrapper. But the component's host delegates the label and commands to the inner dialog inside the Shadow DOM. The modal pattern can display the consuming developers content and the ARIA references across the shadow boundary, workarounds needed to cross the DOM boundary were to add the label data as an attribute to the component, the shadow host, and pass it on to the dialog inside the Shadow DOM programmatically. To activate the modal window I added an eventlistener for a custom command that could pass the command over the boundary.

One requirement that initially could not be satisfied is initial focus placement. The native delegatesFocus-option delegates focus into the Shadow DOM, but does not reach back out to slotted content. The solution found in the Web Awesome component library resolves

this by querying the shadow host for an autofocus attribute, which searches the light DOM from the host element rather than within the shadow tree. This crosses the DOM boundary in a way that is supported consistently. The tradeoff is that correct focus behavior depends on the consuming developer marking the intended element with autofocus - the component cannot determine this automatically. This is a characteristic pattern in shadow DOM accessibility workarounds; what would be implicit or automatic in a standard HTML element must be made explicit through developer convention.

Automated Tests	**Result**	**Relevant findings**
Opening and closing	Pass	Dialog opens using pointer, Enter or Space and closes using Esc or close button
Initial focus, and focus restoration	Pass	Initial focus required adaptation for the slotted light DOM content.
Keyboard focus navigation	Pass	Tab and Shift + Tab could navigate the dialog content without reaching underlying page content. Browser control focus traversal required manual verification in Firefox due to Playwright limitation.
Axe - Closed dialog Ruleset: WCAG 2A/AA, 2.1 A/AA	No violations	No violations were detected by axe-core.
Axe - Open dialog Ruleset: WCAG 2A/AA, 2.1 A/AA	No violations	No violations were detected by axe-core.
Lighthouse	100 Accessibility score	-

Table 4.1.3.1 Condensed table of automated and static tests for Modal pattern. Full automated test implementation is available in the repository [modal.spec.js, test-results]

Manual Tests	Chrome + VO	Firefox + VO	Safari + VO	Observations
Dialog role and accessible name	Pass	Pass	Pass	All announced dialog, accessible name and the focused paragraph
Focus moves to intended element	Pass	Pass	Pass	Intended landing element receives autofocus attribute from the consuming developer
Tab and Shift + Tab remain in modal focus loop	Pass	Pass	Pass	Browser controls may remain in focus loop, underlying content was not reached
Underlying page can not be interacted with	Pass	Pass	Pass	Native dialog in modal mode made background content unavailable
Closing of dialog	Pass	Pass	Pass	If button is used with pointer click: Chrome and Firefox did not display a focus indicator, even though focus returned to the invoking element

Table 4.1.3.2 Condensed table of manual tests for Modal pattern. Full manual test procedure is available in the repository [manual-tests-modal.md]

The final implementation satisfied the modal requirements evaluated in this study, across the tested browser and VoiceOver combinations. Automated testing found no axe-core violations, and Lighthouse returned an accessibility score of 100. The complete automated and manual test implementations and recorded results are available in the project repository.

4.2 Combobox Pattern

A Combobox is an input that controls a popup list of values for the end-user to select from. Comboboxes can be editable, in this case the user types the first letters to narrow down the selectable values based on the initial characters. When navigating through the selectable options the DOM focus remains on the combobox, the options active state is communicated via a aria-activedescendant ID reference.

In a Shadow DOM context it is likely that the consuming developer wants to insert the selectable values through a slot, this will create several reference problems over the DOM boundary.

4.2.1 Requirements for Combobox Pattern

The implementation is based on WAI-ARIA APG Editable Combobox with List Autocomplete[22].

Requirement area	**Requirements**
Roles	combobox, listbox, option
Popup state for listbox	aria-expanded, aria-controls
Autocomplete	aria-autocomplete=”list”
Active option	DOM focus remains on combobox input, active option is communicated with aria-activedescendant via the combobox
Selected	Active option has aria-selected=”true”
Keyboard interaction	Arrow up, arrow down or text input opens listbox, Arrow keys navigate, Enter selects, Escape closes

Table 4.2.1.1 Requirements for the Combobox pattern

4.2.2 Implementation of Combobox Pattern

This Web Component [23] accepts slotted content in the form of list items. They dynamically receive IDs, role and state, whereafter they are slotted into the listbox. The textbox and listbox are implemented in the shadow DOM. This structure both represents how a reusable component could be modelled, and it exposes the DOM-boundary challenges we wish to investigate.

The APG pattern uses an ID reference through *aria-activedescendant* to communicate which option is currently active, this will let a screen reader user know which item they are selecting. Since the list items are slotted in from the light DOM these IDs are not referenceable for ARIA inside the shadow DOM. When setting the list items ID as the combobox’s active descendant the ID reference is resolved in the combobox input’s DOM, the Shadow DOM, and the ID from the slotted option from the Light DOM can not be found.

This creates a new problem, we have to reach into the Light DOM from the Shadow DOM this time.

The elements could be cloned and placed inside the Shadow DOM, where ARIA can reach all IDs, but that goes against the concept of slots and is a confusing pattern. A better solution is to use *ariaActiveDescendantElement* [24] of the Element API. Since we can access the list item element itself through the slot we can use the element as the active descendant reference without using its ID. This will work as long as we refer to an element in the same - or in our case - the parent DOM [25]. The Element API's ariaActiveDescendantElement property is new but has been considered Baseline since April 2025.

4.2.3 Evaluation of Combobox Pattern

The Web Component successfully slots in the content in the form of li-elements populated with values. The consuming developer provides a label value through an attribute on the host element. This value gets passed across the boundary and is used to label the combobox and its associated button. The new problem of reaching out from the shadow DOM was also successfully resolved by using a reflected element-reference property - ariaActiveDescendantElement.

Safari and VoiceOver initially appeared to resolve the cross boundary aria-activedescendant reference; the active option's value and position was announced when using keyboard navigation. Further testing [26] showed that the announcements were triggered by the changes to aria-selected. Assigning a fixed value to aria-active-descendant did not change the announcements, while removing aria-selected stopped the option and position announcements. Therefore the observed behaviour was only a browser/AT difference in what to announce, and not a cross-boundary ID reference.

Two side effects were noted, Chrome in combination with VoiceOver created a *stutter*, it correctly described the active option as e.g "Blue menu item (5 of 7)", but before finishing the message a second message stating "(5 of 7)" interrupted the first one. This is also reproduced when testing the official WAI-ARIA APG code without Shadow DOM and is thus not strictly related to the topic of this thesis, but worth mentioning. A second flaw is that Mozilla Firefox logs a warning: "Empty string sent to getElementById()" when using ariaActiveDescendantElement, this is annoying for developers but no effect on the components usability and accessibility was observed.

Automated Tests	**Result**	**Relevant findings**
Roles, States	Pass	Using a reflected element reference enabled the combobox to refer to the active light-DOM option across Shadow DOM boundary
Keyboard and mouse Interaction	Pass	Opening, navigating, selecting and closing behaved as required
Axe - Closed listbox.	No violations	No violations were detected

Ruleset: WCAG 2A/AA, 2.1 A/AA		by axe-core.
Axe - Open listbox. Ruleset: WCAG 2A/AA, 2.1 A/AA	No violations	No violations were detected by axe-core.
Lighthouse	100 Accessibility score	-

Table 4.2.3.1 Condensed table of automated tests for Combobox pattern. Full automated tests and results are available in the repository [combobox.spec.js, test-results]

Manual Tests	**Chrome + VO**	**Firefox + VO**	**Safari + VO**	**Observations**
Combobox name announced	Pass	Pass	Pass	-
Listbox state announced	Pass	Pass	Pass	Exact announcement wording differed between browsers
Active option announced	Pass	Pass	Pass	Chrome repeats position, which causes a stutter. Not Shadow DOM related
Filtered navigation	Pass	Pass	Pass	Correct filtered options announced
Escape closes listbox	Pass	Pass	Pass	State “Collapsed” was announced

Table 4.2.3.2 Condensed table of manual tests for Combobox pattern. Full manual tests and results are available in the repository [manual-tests-combobox.md]

The final implementation satisfied the Combobox requirements evaluated in this study, across the tested browser and VoiceOver combinations. Automated testing found no axe-core violations, and Lighthouse returned an accessibility score of 100. The complete automated and manual test implementations and recorded results are available in the project repository.

4.3 Checkbox Pattern

A checkbox is most known as a dual-state toggle between two choices. A classic use case is: “Yes I have read and agree to all terms and conditions”. In a real world example they are often associated with a form that can send the selected data to another application. WAI-ARIA also supports a tri-state checkbox with an additional mixed state, but this study examines the more common dual-state pattern.

4.3.1 Requirements for Checkbox Pattern

The implementation is based on WAI-ARIA APG Checkbox Example (Two State) [27]

Requirement area	**Requirements**
Roles	role=”group”, checkbox has “checkbox”
States	aria-checked shows ”false” or “true”
Keyboard	Included in tab order, Toggles checked state with space
Labels	Each checkbox has an accessible name, the group has an accessible name
Study-specific Shadow DOM requirement	
Form participation	Checked elements are included in a Light DOM forms submission values.

Table 4.3.1.1 Requirements for the Checkbox pattern

4.3.2 Implementation of Checkbox Pattern

The pattern uses a container with a role of *group* to semantically related checkboxes within a theme, in the APG patterns case Sandwich Condiments.

Following the APG pattern two Web Components were created[28], an outer grouping container *wc-checkbox-group* and an inner *wc-checkbox* that represent the checkbox itself.

The outer component receives a *group-label* attribute,which is visually rendered as a heading element inside the Shadow DOM, this headline is also referred to with *aria-labelledby* from a sibling div element that is the group container. Inside the group container the wc-checkbox components are slotted in from the Light DOM. Since the wc-checkbox components stem from the Light DOM, the group component can access them via querySelectorAll on the host element itself. This is used to automatically assign a shared name attribute derived from the group label.

Since a checkbox is most commonly used in a form context, each wc-checkbox gets associated to a parent form using *formAssociated = true* and I also attach an ElementInternals object in the constructor via this.attachInternals(). This mechanism allows the custom element to participate in a form in the Light DOM from within Shadow DOM.

Without it, the component's state would be invisible to any surrounding form element. When a checkbox is toggled, *this._internals.setFormValue()* is called with either the label text or *null*, updating the form-associated value accordingly.

The checkbox role, state and keyboard interaction are all applied directly to the host element rather than to an inner element in the Shadow DOM. The role of checkbox, aria-checked status, tabindex and aria-label attributes are set on the custom element itself in the Light DOM, which means they are visible to the accessibility tree without any cross boundary complications. The Shadow DOM in this component contains only the visual representation: a span element set to aria-hidden that displays the label text, and the CSS for the checkbox appearance including the SVG-based checked/unchecked indicator rendered via ::before on the :host.

Toggle behavior is handled through *click* and *keyup* listeners. A *keydown-listener* separately handles the Space key to prevent the default page scroll behavior, while the actual toggle fires on *keyup*, consistent with the APG specification.

The visual styling of the elements using *:host* combined with attribute selectors like *:host([aria-checked="true"])* works cleanly from inside Shadow DOM. The focus and hover styles are similarly applied via :host(:focus) and :host(:hover), this allows visible focus indicators to be applied. These mechanisms allow the component author to apply styling to the host that could support meeting WCAG 2.1 AA standards, such as Success Criterion 2.4.7 - visible focus - crossing the Shadow DOM boundary limitations.

4.3.3 Evaluation of Checkbox Pattern

The checkbox pattern proved to be the most straightforward of the three patterns to implement in a Shadow DOM context. Due to the placing of the ARIA semantics on the host element rather than inside the Shadow DOM, the accessibility tree has direct access to the relevant roles and states without any cross-boundary ID references being required.

Form association, which represents the primary Shadow DOM challenge for this pattern, was resolved through the ElementInternals API. The formAssociated flag and setFormValue() allow the component to submit its value as part of a Light DOM form, which would otherwise be impossible since form elements cannot natively reach inside a Shadow DOM to collect values.

Overall, the checkbox pattern demonstrates that when ARIA semantics are anchored to the host element and form-participation is handled through ElementInternals, the Shadow DOM encapsulation introduces minimal friction compared to the modal and combobox patterns.

Automated Tests	Result	Relevant findings
Roles, States, Accessible names	Pass	Roles, states and names placed directly on the custom-element hosts, avoiding cross-boundary references.
Keyboard and mouse Interaction	Pass	Navigating with Tab, checking and unchecking worked as required
Light DOM form participation	Pass	ElementInternals allowed the checked boxes to add values to the Light DOM form
Axe Ruleset: WCAG 2A/AA, 2.1 A/AA	No violations	No violations were detected by axe-core.
Lighthouse	100 Accessibility score	-

Table 4.3.3.1 Condensation of automated and static tests for Checkbox pattern. *Full automated tests and results are available in the repository [checkbox.spec.js, test-results]*

Manual Tests	Chrome + VO	Firefox + VO	Safari + VO	Observations
Accessible name announced when navigating into the group	Pass	Pass	Pass	Group name was announced when tabbing into the group
Checkbox name, role and state announced	Pass	Pass	Pass	Exact announcement wording differed between browsers
Keyboard interaction	Pass	Pass	Pass	Focus order and checked/unchecked state was announced

Table 4.3.3.2 Condensation of manual tests for Checkbox pattern. *Full manual tests and results are available in the repository [manual-tests-checkbox.md]*

The final implementation satisfied the Checkbox requirements evaluated in this study, across the tested browser and VoiceOver combinations. Automated testing found no axe-core violations, and Lighthouse returned an accessibility score of 100. The complete automated and manual test implementations and recorded results are available in the project repository.

4.4 Pattern Analysis

Based on the findings from these three implementations we can answer the questions based on empirical findings.

4.4.1 RQ1 - How does the selected WAI-ARIA APG patterns work when implemented in Shadow DOM-based Web Components?

The Evaluations in 4.1-4.3 show that the implemented WAI-ARIA APG patterns are conceptually applicable in Shadow DOM-based Web Components, but with different levels of adaptation.

The implementation of the Modal Dialog pattern required delegation of data through the shadow host to overcome DOM boundary limitations. Since the Invoker Commands API depends on ID references, a button in the Light DOM could not directly target a dialog element inside the Shadow DOM. This required delegation through a custom command handled by the host component. Focus management required additional implementation work, the native focus delegation is good for the content in Shadow DOM but did not consistently reach the slotted content from Light DOM across all browsers. A workaround based on the autofocus attribute was required to achieve reliable initial focus behavior.

The APG combobox pattern relies heavily on IDREF-based ARIA relations, particularly aria-activedescendant, which relies on that the referenced option exists within the same DOM scope. Since list options were supplied through slots from the Light DOM, these references failed in the first iteration. A successful implementation was only achieved through the newer ariaActiveDescendantElement API, which allows direct element references rather than string-based IDs.

The Checkbox pattern's accessibility features like role, aria-checked, and keyboard interaction were placed directly on the host element in the Light DOM, this avoided most of the communication over the Shadow DOM boundary. The primary challenge concerned form participation, which was resolved using the ElementInternals API.

Taken together, these observations suggest that the APG patterns are designed on the assumption of a single DOM tree where ID-based relations, focus behavior, and semantic associations are freely available.

Shadow DOM encapsulation breaks this assumption by separating components into isolated DOM scopes. The three evaluated patterns demonstrated different levels of implementation difficulty.

Overall, the results indicate that the selected APG patterns are implementable but not directly transferable to Shadow DOM environments. Patterns relying on IDREF relationships and cross-boundary interaction require additional implementation strategies, while patterns whose attributes can be anchored to the shadow host adapt more naturally.

4.4.2 RQ2 - Which accessibility problems occur when working with shadow DOM encapsulation, what are the technical causes?

The study identified four recurring categories of accessibility problems caused by Shadow DOM encapsulation: Cross-boundary ID limitations, focus management issues, form participation constraints, and browser- or assistive technology-inconsistencies.

The most common issue was the inability to perform cross-boundary ID references. Many accessibility relationships depend on IDs, such as aria-labelledby, aria-controls, and aria-activedescendant. When the referencing element and its intended target are located in separate DOM scopes these relations could not be solved through the standard ID-based approach.

A second problem concerned focus management. Native focus delegation did not reach slotted Light DOM content consistently. In the modal dialog implementation this prevented reliable initial focus placement and required an implementation strategy where initial focus is set by the consuming developer.

A third issue was form participation limitations. This challenge emerged in the checkbox implementation, where the component state required explicit form association through ElementInternals to contribute to the parent form in the Light DOM.

Finally, browser and assistive technology inconsistencies were observed throughout testing. The wording and behaviour of announcements differed between browser and VoiceOver combinations, some behaviours required additional testing to discern whether they were caused by Shadow DOM or something else.

The interpretation is that the collected observations indicate that the primary technical cause is the separate encapsulation introduced by theShadow DOM. Accessibility mechanisms that depend on relationships, focus behaviour or form participation across this DOM boundary often require additional implementation strategies. Discrepancies in how browsers and assistive technology handle this boundary can further complicate the evaluation of these behaviours.

4.4.3 RQ3 - Which implementation strategies and workarounds can be used to support WCAG 2.1 AA compliance for Web Components when APG patterns can not be directly applied?

The findings show that implementation strategies that can support developers in reaching WCAG 2.1 AA conformance for Shadow DOM-based Web Components exist. Five viable strategies emerged in this study: shadow host delegation, attribute forwarding, ElementInternals, reflected element references, and anchoring accessibility semantics on the host.

Shadow host delegation was used when interactions could not directly cross the DOM boundary. In the modal dialog pattern, commands originating in the Light DOM were routed through the shadow host and programmatically delegated to elements inside the Shadow DOM. This enabled interaction patterns that would otherwise fail due to inaccessible ID references.

Attribute forwarding was used to transfer accessibility information across DOM boundaries. For example, labels provided as attributes on the custom element host were programmatically forwarded to internal Shadow DOM elements. This allowed ARIA semantics to be preserved despite encapsulation.

The ElementInternals API was used in the checkbox implementation. It enabled form

participation through setFormValue(), allowing custom elements to interact with Light DOM forms.

In the combobox implementation reflected element references were used, the ariaActiveDescendantElement property enabled direct element references across compatible DOM scopes, avoiding broken IDREF relationships.

A further strategy involved anchoring semantics on the shadow host rather than inside the Shadow DOM whenever possible. The checkbox pattern demonstrated that when roles, states, and keyboard behavior are exposed directly on the host element, accessibility barriers are significantly reduced.

Overall, the findings suggest that the accessibility requirements evaluated in this study can be supported in Shadow DOM-based Web Components, but developers must often adapt APG patterns through explicit architectural adaptations rather than relying on standard unchanged implementations.

This study also found limitations to the aforementioned strategies. Many solutions rely on relatively new APIs, browser behaviour is sometimes inconsistent and some strategies shift responsibility to the developer consuming the component, rather than being solved internally by the component itself.

5. Discussion

The study exposes the duality of Web Components, the encapsulation that makes the components modular and reusable also causes accessibility challenges. The introduction of Shadow DOM protects the components from being affected by surrounding scripts and styles but also hinders the accessibility mechanics that need access to the internals of the component. Many current accessibility patterns rely on unrestricted DOM access.

5.1 The Consuming Developer

A recurring finding was that several accessibility requirements could not be fully handled inside the Web Component itself, but depended on correct handling by the consuming developer.

However, this is not unique to web components. Native HTML elements also rely on correct handling. An image requires meaningful alt text, list elements require the right child elements and form inputs have their own system of associating labels and inputs. In this sense a reusable accessible Web Component can be compared to offering a usage contract, preferably with documented expectations and rules.

The modal dialog can be used as an example; the correct initial focus depends on the consuming developer adding the autofocus attribute to the intended focal target. While this introduces some responsibility for the consuming developer it also allows flexibility while following established HTML syntax.

Accessibility therefore does not only depend on the component itself, but also on clearly communicating the accessibility requirements of the component's public interface.

5.2 EAA Compliance

The findings indicate that Shadow DOM does not inherently prevent developers from implementing Shadow DOM-based Web Components that conform to the WCAG 2.1 AA

requirements. However, the evaluated patterns show that meeting such requirements can require additional implementation strategies compared to implementations that do not cross DOM boundaries.

This study does not establish full WCAG 2.1 AA conformance or legal compliance with the EAA. Instead, it identifies technical barriers introduced by the Shadow DOM and demonstrates strategies that can aid developers in crossing some of these barriers.

Given the legal obligations introduced by the EAA, these findings could be relevant to developers building services with Web Components, as the implementation strategies identified in this study address accessibility barriers where formal guidance is limited.

5.3 Limitations of the Study

Several limits should be considered when reading this study. First, only three APG patterns have been evaluated, so the results cannot be generalized to all patterns. However the patterns were selected to represent a broad set of accessibility challenges, other patterns could introduce new issues.

Second, the implementation and evaluation were conducted by a single developer, which introduces potential bias in both implementation choices and interpretation of results.

Third, testing was limited to modern versions of Chrome, Firefox and Safari together with Apple VoiceOver and keyboard navigation. Other assistive technologies, including screen readers on Windows and refreshable braille displays, were outside the scope of this study.

Finally, the study focused on technical accessibility behaviour and implementation requirements derived from the APG pattern, rather than usability testing with people with disabilities. A technically correct implementation may still present practical usability challenges in real-world use.

6. Conclusion & Future Work

Web Components and the surrounding accessibility ecosystem continue to evolve rapidly. Ongoing developments such as Cross Root ARIA may fundamentally change how accessibility relationships can be expressed across DOM boundaries in the future. Therefore, the findings of this study must be understood in the context of the current web platforms capabilities.

This study investigates how selected WAI-ARIA APG patterns could be implemented as Shadow DOM-based Web Components, with particular emphasis on accessibility problems introduced by the DOM boundary and strategies to address them.

The findings showed that the selected APG patterns are conceptually transferable to Shadow DOM environments, but generally cannot be implemented directly without adaptation because the patterns assume a single DOM structure.

The primary accessibility challenges introduced by Shadow DOM were cross-boundary ID limitations, focus management complexities, form participation constraints, and browser inconsistencies.

Several practical implementation strategies were identified, including shadow host delegation, attribute forwarding, ElementInternals, reflected element-reference and placing accessibility semantics on the shadow host whenever possible.

This study demonstrates concrete implementations of three APG patterns as Shadow DOM based Web Components. It documents both the accessibility issues when working across the DOM boundary and the practical implementation strategies that can be used to overcome the limitations introduced by the encapsulation.

Future work could expand the study to additional APG patterns, investigate the effects of Web Component frameworks such as Lit on accessibility workflows, and evaluate implementations using a broader set of assistive technologies and real users with disabilities. Of particular interest is the future impact of the Cross Root ARIA proposals and further implementation of ARIA Element Reflection, which may significantly reduce the need for current workarounds and alter best practices for accessible Web Components.

References

[1] ”European Accessibility Act (EAA) - European Commission”, European Commission - Directorate-General for Communication. Accessed: Sep. 20, 2026 [Online] Available: https://commission.europa.eu/strategy-and-policy/policies/justice-and-fundamental-rights/disability/european-accessibility-act-eaa_en

[2] "Web Content Accessibility Guidelines (WCAG) 2.1", World Wide Web Consortium. Accessed: Sep. 20, 2026 [Online]. Available: https://www.w3.org/TR/WCAG21/

[3] “ARIA Authoring Practices Guide | APG | WAI | W3C”, World Wide Web Consortium. Accessed: Sep. 20, 2026 [Online] Available: https://www.w3.org/WAI/ARIA/apg/

[4] “Chrome Platform Status”, Google. Accessed: Sep. 04, 2026 [Online] Available: https://chromestatus.com/metrics/feature/timeline/popularity/804

[5] W. Johnson, E. Marquez, M. Warren, “Web Components Community Group: 2023 Spec/API status”, Accessed: Sep. 20, 2026 [Online] Available: https://w3c-cg.github.io/webcomponents-cg/2023.html

[6] L. Balter, "Cross Root ARIA Delegation Explainer", GitHub. Accessed: Feb. 04, 2026 [Online] Available: https://github.com/leobalter/cross-root-aria-delegation/blob/main/explainer.md

[7] “site:stackoverflow.com shadow dom a11y - Google Search”, Google. Accessed: Feb. 08, 2026 [Online] Available: https://www.google.com/search?q=site%3Astackoverflow.com+shadow+dom+a11y&oq=site%3Astackoverflow.com+shadow+dom+a11y&gs_lcrp=EgZjaHJvbWUyBggAEEUYOTIGCAEQRRg60gEJMjM1MDhqMGo3qAIAsAIA&sourceid=chrome&ie=UTF-8

[8] A. A. Gregory, “Developing Accessible Web Components”, theses.fh-hagenberg.at, Accessed: Sep. 20, 2026 [Online] Available: https://theses.fh-hagenberg.at/thesis/Gregory19

[9] J. White, M. Hakkinen, J. Grant, “Meeting Accessibility Challenges with

Web Components”, Journal on Technology and Persons with Disabilities, [Online] Available: https://scholarworks.calstate.edu/concern/file_sets/5999n688m

[10] A. Parab, “Web Components - The LEGO Bricks of Modern Web Development”, [Online] Available: https://sarcouncil.com/2025/07/web-components-the-lego-bricks-of-modern-web-development

[11] N. Lawson, “Shadow DOM and the problem of encapsulation”, Read the Tea Leaves, [Online] Available: https://nolanlawson.com/2023/12/30/shadow-dom-and-the-problem-of-encapsulation/

[12] “Web Components, Design Systems and Accessibility”, youtube.com. Accessed: Feb. 08, 2026 [Online] Available: https://www.youtube.com/watch?v=xz8yRVJMP2k&t=1917s

[13] “Web Components Community Group API and Specs Report 2022 - TPAC 2022 breakout – 14 September 2022”, W3.org. Accessed: Feb. 04, 2026 [Online] Available: https://www.w3.org/2022/09/14-webcomponent-cg-minutes.html

[14] A. R. Hevner, et al. Design science in information systems research. MIS quarterly (2004): 75-105. https://doi.org/10.2307/25148625 https://wise.vub.ac.be/sites/default/files/thesis_info/design_science.pdf

[15] “<dialog>: The Dialog element”, Mozilla Developer Network. Accessed: Feb. 08, 2026 [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/dialog

[16] “Invoker Commands API – Browser compatibility”, Mozilla Developer Network. Accessed: Feb. 08, 2026 [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Invoker_Commands_API#browser_compatibility

[17] “Understanding Success Criterion 2.1.2: No Keyboard Trap”, World Wide Web Consortium. Accessed: Feb. 08, 2026 [Online]. Available: https://www.w3.org/WAI/WCAG21/Understanding/no-keyboard-trap.html

[18] “Clarification regarding focus trapping in modal dialogs”, World Wide Web Consortium GitHub Issues, issue #4185. Accessed: Feb. 08, 2026 [Online]. Available: https://github.com/w3c/wcag/issues/4185

[19] S. Darsell, “wc-modal.js”, GitHub repository. Accessed: Sep 20, 2026 [Online]. Available: https://github.com/SIMON-5000/apg-wc-test/tree/main/src/components/modal

[20] “Invoker Commands API”, Mozilla Developer Network. Accessed: Sep. 20, 2026 [Online]. Available https://developer.mozilla.org/en-US/docs/Web/API/Invoker_Commands_API

[21] “dialog.ts”, Web Awesome GitHub repository. Accessed: Feb. 08, 2026 [Online]. Available: https://github.com/shoelace-style/webawesome/blob/next/packages/webawesome/src/components/dialog/dialog.ts

[22] “Editable Combobox With List Autocomplete Example | APG | WAI | W3C”, World Wide Web Consortium Accessed: Sep. 20, 2026 [Online]. Available https://www.w3.org/WAI/ARIA/apg/patterns/combobox/examples/combobox-autocomplete-list/

[23] S.Darsell, “wc-combobox.js”, GitHub repository. Accessed: Sep. 20, 2026 [Online]. Available: https://github.com/SIMON-5000/apg-wc-test/tree/main/src/components/combobox

[24] “ARIA: ariaActiveDescendantElement”, World Wide Web Consortium. Accessed: Sep. 18, 2026 [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Element/ariaActiveDescendantElement

[25] “Reflected element reference scope”, Mozilla Developer Network. Accessed: Feb. 08, 2026 [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Reflected_attributes#reflected_element_reference_scope

[26] S.Darsell, “manual-tests-combobox”, GitHub repository. Accessed: Sep. 20, 2026 [Online]. Available: https://github.com/SIMON-5000/apg-wc-test/blob/main/test-results/manual-tests/manual-tests-combobox.md

[27] “Checkbox Example (Two State) | APG | WAI | W3C”, World Wide Web Consortium Accessed: Sep. 20, 2026 [Online]. Available https://www.w3.org/WAI/ARIA/apg/patterns/checkbox/examples/checkbox/

[28] S.Darsell, “wc-checkbox-group.js”, GitHub repository. Accessed: Sep. 20, 2026 [Online]. Available: https://github.com/SIMON-5000/apg-wc-test/tree/main/src/components/checkbox

Appendices

Appendix A:

Testing environment

Playwright 1.60.0, utilizing browsers versions:

- Chromium 148.0.7778.96
- Mozilla Firefox 150.0.2
- WebKit 26.4

@axe-core/playwright 4.11.2
Lighthouse 13.4.0

Manual testing:

Browser	**Version**
Chrome	148.0.7778.216 (Official Build) (arm64)
Firefox	151.0.2 (aarch64)
Safari	26.5 (21624.2.5.11.4)

Assistive technology: Voice Over Version 10 (993)